

RESEARCH THE FOLLOWING QUESTIONS ON FILE HANDLING.

1. How does Python handle file operations (read, append, delete) across different file formats such as XML, JSON, and TXT?
 - Python uses built-in functions and libraries to manage files.
 - TXT : `open()`
 - JSON : `json` module
 - XML : `ElementTree`

Example:
`file = open("data.txt", "r")`
`print(file.read())`

Python can easily read, write, append, and delete data using different modules depending on the file type.
2. What are the differences in performance when manipulating structured (XML, JSON) vs unstructured (TXT) data?
 - TXT files are faster because they are simple. JSON and XML are slower because they require parsing.

Example:
TXT- Notes or messages
JSON - User profiles in apps
XML - Configuration files

Structured files are easier to organize but slower to process than TXT files.
3. Which file format is more efficient for storing and modifying data in Python applications?
 - JSON is usually more efficient because it is lightweight and easy to edit.

Example:
Social media apps use JSON to store user information. JSON is widely used because it balances speed and structure.
4. What are the common challenges when performing CRUD operations in different file types?

Challenges include:

 - Corrupted files
 - Missing data
 - Invalid format
 - Slow performance

Example:
A broken XML tag may stop the system from reading the file properly. Different file types require different handling methods.
5. How does Python read data from a plain text (.txt) file?
 - Python uses `open()` with read mode "r".

Code Example:
`file = open("notes.txt", "r")`
`print(file.read())`

Real-Life Example:

A diary application reads saved notes from a TXT file. The content is read as plain text.

6. What methods are used to append new data into a TXT file?

- Append mode "a" is used.

Code Example:

```
file = open("notes.txt", "a")
file.write("New Note")
```

Real-Life Example:

A logging system adds new records at the end of a log file. Appending keeps old data while adding new content.

7. How can specific lines or content be deleted from a TXT file?

- The file is read first, unwanted lines are removed, then the file is rewritten

Example:

Removing a completed task from a to-do list. Python cannot directly delete a specific line, so the file must be updated manually.

8. What are the limitations of using TXT files for structured data storage?

- TXT files limitations include:
- Lack structure
- Cannot store nested data easily

Are harder to search

Example:

Storing employee records in TXT makes sorting difficult. TXT files are better for simple notes rather than databases.

9. How does file size affect reading and writing speed in TXT files?

- Larger files take longer to process.

Example:

A 1GB log file loads slower than a 1MB file. Big files consume more memory and processing time.

10. How does Python parse and read data from a JSON file?

- Python uses the json module.

Code Example:

```
import json
```

```
with open("data.json", "r") as file:
    data = json.load(file)
```

Real-Life Example:

An online shopping app reads product data from a JSON file. JSON data becomes a Python dictionary after parsing.

11. What is the process of appending new objects to a JSON file?

- Load existing data, add new data, then save the file again.

Example:

Adding a new customer account to a customer database. JSON must remain properly formatted after modifications.

12. How can specific key-value pairs be deleted in a JSON structure?

- Use the del keyword.

Code Example:

```
del data["age"]
```

Real-Life Example:

Removing an old address from a user profile. The selected key and value are removed from the JSON object.

13. What advantages does JSON provide over TXT in data handling?

Advantages:

- Organized structure
- Easier searching
- Supports nested data

Example:

A mobile app stores settings in JSON format. JSON is easier to process compared to plain text.

14. How does Python ensure data integrity when modifying JSON files?

- Python validates the structure before saving changes.

Example:

Checking if all required fields are present before updating data. Validation prevents corrupted or incomplete JSON files.

15. How does Python read and parse XML files using libraries like ElementTree?

- Python uses the ElementTree library.

Code Example:

```
import xml.etree.ElementTree as ET
```

```
tree = ET.parse("data.xml")  
root = tree.getroot()
```

Real-Life Example:

A company system reads employee records stored in XML format. XML data is organized into elements and tags.

16. What are the steps to append new elements to an XML document?

Steps to append new elements to an XML document:

- Read XML file
- Create new element
- Add element
- Save file

Example:

Adding a new student record to an XML database. New elements must follow proper XML structure.

17. How can nodes or elements be deleted from an XML file?

- Use the `.remove()` method.

Code Example:

```
root.remove(child)
```

Real-Life Example:

Deleting inactive employee records from an XML file. Removing incorrect nodes may damage the XML structure.

18. What are the differences between XML and JSON in terms of data manipulation?

- XML is more complex
- JSON is simpler and faster

Example:

JSON is commonly used in APIs, while XML is used in older enterprise systems. JSON is easier to edit programmatically.

19. How does XML structure affect processing time in Python?

- Complex XML structures increase parsing time.

Example:

A large XML file with many nested tags takes longer to load. More tags mean more processing work

20. Which is faster: reading data from XML, JSON, or TXT files in Python?

Fastest to slowest:

- TXT
- JSON
- XML

Example:

Opening a TXT note is faster than loading a large XML document. XML processing is slower because of its detailed structure.

21. Which format is easier to modify programmatically and why?

- JSON is easier because it uses simple key-value pairs.

Example:

Editing user settings in a mobile app. JSON is readable and easier for developers to manipulate.

22. How does memory usage differ when handling XML, JSON, and TXT files?

- TXT has lowest memory usage
- JSON- Moderate
- XML- Highest

Example:

Large XML files consume more RAM than TXT files. XML stores additional tags, increasing memory usage.

23. Which file format is more suitable for large-scale applications?

- JSON is more suitable for modern applications.

Example:

Web APIs and cloud applications commonly use JSON. JSON is lightweight and works well with web technologies.

24. What are the security considerations when modifying these file types?

Important security considerations are the following:

- Validate user input
- Prevent unauthorized access
- Avoid corrupted files
- Create backups

Example:

A hacker inserts malicious data into a JSON file. Security measures protect files from corruption and attacks.

Instructions:

Please provide the examples of every question, explain and discuss.

Submit on May 7, 2025 @ 12 noon. @ GClass.